

Concordance

A Power BI semantic model states thousands of facts about itself. This reads them, writes the requirements documents from them, and translates each measure into SQL — or explains precisely why it cannot.

1,274 tests passing, 64 files	3 input formats read	5 SQL dialects emitted	22 DAX functions refused, with reasons
54k lines across app, UI and tests			

01 The idea

A measure has no value until a filter context is named.

That single sentence is the whole design. Power BI's `Net Sales` is not a number; it is a calculation waiting to be told which rows it applies to. Ask it over a whole file and you get 1.2M. Ask it on a page pinned to June and you get 387K. Both are correct answers to different questions, and a tool that shows one without naming the question is not documenting the model — it is misreporting it.

So every figure here travels with the query that produced it and the filter it was computed under. Where a measure has no single answer, no number is invented; the reason is given in terms of the report, because that is the thing the reader has actually seen.

The same rule applied to reading. A model declares far more about itself than a schema dump shows — how each measure should be rendered, which column orders another, which table is the calendar, which column holds a latitude. Every one of those was being inferred or guessed at some point in this project's history, and every one is now read from the file. What is still not read is listed in §06 with the reason attached.

02 Architecture

Three kinds of source collapse into one shape, and everything downstream is written once.

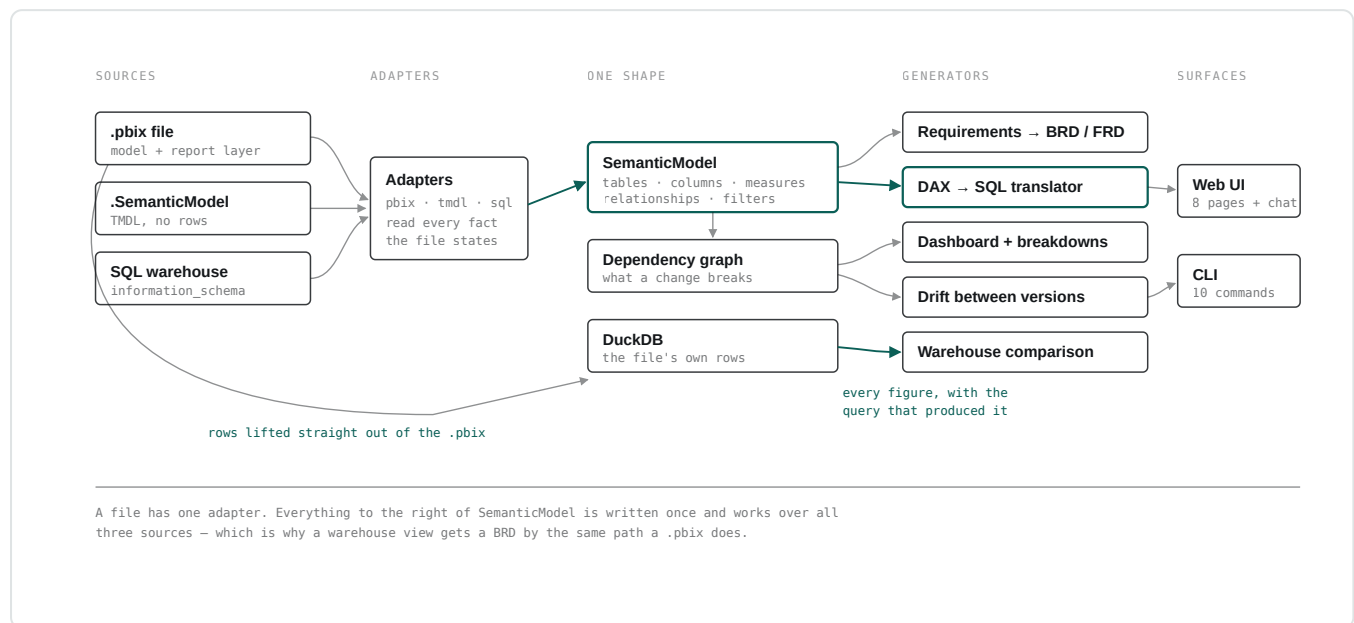


Figure 1. The single normalisation point is the whole architecture. Adapters absorb format differences; nothing downstream knows or cares which file it came from. The teal path carries the discipline: the file's own rows go into DuckDB, and every figure comes back out with the query that produced it.

03 The pipeline

What happens between opening a file and reading a document.

read	The adapter pulls every declaration the file makes — not just tables and formulas, but format strings, sort-by columns, data categories, hidden flags, descriptions, storage mode, and the report's own filters at report, page and tile scope.
normalise	All of it lands in one <code>SemanticModel</code> . Each object is fingerprinted over the thing that defines it, so an edit to a formula changes its identity and an edit to whitespace does not.
relate	A dependency graph is built across measures, columns, tables and drill paths — the basis for impact analysis and for pointing at the requirements a change puts in question.
compute	The file's own rows are loaded into an in-memory DuckDB. Every figure shown anywhere is produced by running a query , never copied from the report.
translate	Each measure's DAX is lexed into an AST and rendered as SQL at a stated grain — or refused with a reason . Five dialects.
derive	Requirements are generated, each bound by fingerprint to the object it came from and marked with whether the model states it or the tool inferred it.
surface	Eight web pages, a chatbot with tools over the model, and ten CLI commands — all reading the same structure.

04 What you can open

Eight pages, each answering a different question. Fuller descriptions in the accompanying guide.

Overview

THE FRONT DOOR

What this file holds, and what could not be read from it. Points a reviewer and a maintainer at different tabs.

Dashboard

THE REPORT, EXPLAINED

The report's tiles with the DAX and SQL behind each. Figures computed live, with charts, cross-filtering and a real map.

Dataset + SQL

EVERYTHING AT ONCE

Every measure, its DAX, and the equivalent SQL at a grain you choose. Joins included.

BRD & FRD

THE DOCUMENTS

Generated requirements, each bound to its evidence and marked with how it was arrived at. Word or Markdown.

Browse objects

THE MODEL ITSELF

Tables, columns, measures and hierarchies as a tree, with impact analysis on any object.

Drift

WHAT CHANGED

Two versions compared, and the requirements a change puts in question.

Warehouse

DOES THE DATABASE AGREE?

A Power BI measure against a warehouse view, structurally compared. Three verdicts.

To confirm

THE QUEUE

Everything inferred rather than read, waiting on a person's decision.

05 Principled refusals

The most important thing this tool does is decline to answer.

Twenty-two DAX functions stop a translation. A measure using `ALL`, `ISINSCOPE`, `SELECTEDVALUE`, `RANKX`, `TOTALYTD` or their relatives has no fixed answer — its result changes with what the user has clicked or which period the report is showing. Emitting a query anyway would produce a number that looks authoritative and is wrong.

SAMPLE MODEL	MEASURES	COMPUTED	WHY THE REST ARE REFUSED
Sales & Returns	58	6	17 use <code>ALL</code> ; 12 are per-period only; 7 use <code>ISINSCOPE</code>
Store Sales	32	29	one <code>ALL</code> ; one constant; one ambiguous column name
Supply Chain	4	4	—

6 of 58 reads as a failure and is not. Microsoft's Sales & Returns is a demonstration of interactive DAX, built almost entirely from those functions. A model written for reporting rather than for demonstration behaves like Store Sales.

Eight time-shift patterns that *are* unambiguous — previous month, previous quarter and their kin — were originally refused too. A reviewer called that out as over-cautious and was right: such a measure names its own grain, so it is translated at that grain and nowhere else.

06 Limitations

Everything this tool does not do, labelled by kind. A deliberate refusal is not the same as an unbuilt feature, and neither is the same as a blocked environment.

Joins keep unmatched rows unless the file says not to **BY DESIGN**

A Power BI relationship does not discard rows, so a translation joins outward from the table being aggregated and keeps its rows; a group with no value is labelled (Blank) . Where the author assumed referential integrity, the join is inner, because the file has said those rows cannot exist.

The cost of getting it wrong is not a missing row: with one orphaned sale worth 700 of 1,000, an inner join reported the total as 1,000, the parts as 300, and then announced the measure as an average or a ratio — an explanation of the tool's own dropped row, offered as a fact about the measure.

Figures are a snapshot, and the document dates it **BY DESIGN**

Every number comes from the rows stored inside the file. That is what makes it checkable; it is also as old as the last refresh, which the documents now state. A .SemanticModel folder carries no rows at all, so it produces documents and browsing but no figures. A DirectQuery table has no rows in the file either — those tables are reported by name.

Three kinds of number format are not applied **BY DESIGN**

Date pictures, the trailing comma that divides by a thousand, conditional formats beyond three sections, and formats that hide negatives. Nine of the twelve distinct formats across the sample files render; the three declined are all date pictures.

Why: a half-applied format changes the number, not its appearance. Where one is declined the raw figure is shown — always true, even when it is not what Power BI would draw.

Translated object names are counted, not attached **NOT BUILT**

Each translated name is keyed by a bare integer whose meaning depends on the object type, and the reader publishes no mapping for that enum.

Why not guessed: calling one measure by another's French name is worse than saying the translations were not read.

Some row-level security detail is invisible to the reader READER LIMIT

Table filters inside a role are read in full, but a role that filters no table produces no row in the source, and model-level permissions are never exposed. Stated wherever security is reported.

Q&A synonyms are read and deliberately unused BY DESIGN

This looked like the best remaining source for a business glossary. Reading them settled it: across the three sample files all 565 terms are machine-generated — the object's own name respaced, or a thesaurus suggestion. Power BI proposes “artifact advertised” for `Product Advertised`.

Why: a glossary where a reader cannot tell which entries a person stood behind is worse than no glossary. The count is reported with the reasoning attached.

Layout, theming and drill-through bindings are not read BY DESIGN

Tile positions, sizes, colours, fonts and mobile layouts are outside scope — this documents what a report says, not how it is arranged.

The genuine gap: drill-through page bindings. A page reached by drilling through always shows its figures under a filter passed in from another page, and that is a filter context this does not yet name. It is the clearest next thing to build.

Snowflake and Databricks are not verified live ENVIRONMENT

The SQL path runs end to end against DuckDB, whose information schema follows the same standard both implement, and the Snowflake connector is unit-tested against a fake cursor — proving the query text, case handling and wiring.

What that cannot prove: a real handshake. This sandbox blocks outbound connections to `*.snowflakecomputing.com` at the network policy layer, confirmed against a real trial account rather than assumed.

Warehouse comparison is structural, not semantic BY DESIGN

DAX and SQL never hash alike even when they compute the same number, so what is compared is what each definition reads. Proving two arbitrary expressions equivalent is undecidable in general — which is why “needs review” is a real verdict rather than hedging.

Charts have fixed bounds BY DESIGN

At most 10 slices with the remainder folded into a labelled “N more”; a column is offered as a dimension only at 2–30 distinct values; 120 points on a time series, 40 on a sparkline. The map needs coordinates declared in the file — a postcode is not geocoded, because a map drawn from a guess puts a location somewhere it is not.

The chatbot needs a key DEPLOYMENT

Five providers with automatic fallback. With none configured the panel says so rather than degrading into guesswork, and it answers only from tools that read the extracted model.

Power BI Service links await credentials DEPLOYMENT

Opening a model from a direct URL works, with full protection against requests being redirected at internal addresses. The `app.powerbi.com` path is built and waiting for an API key.

07 Verification

What is proven, and what is merely written.

1,274 tests pass across 64 files. Guards are mutation-tested: the guard is broken deliberately and the test confirmed to go red, because a test that passes either way proves nothing. Two source audits run as tests — enumerating everything the file formats offer against everything the readers name — so a source that starts carrying data, or one a reader quietly stops reading, fails the build.

44 tests are skipped, and 43 of them for one reason: `AdventureWorks_Sales.pbix` is not in the repository. It was only ever an upload and was lost when a container reset. The tests are written and run the moment the file is present. The 44th needs an optional dependency for one authentication mode.

Two things are untested rather than unwritten: a live Snowflake handshake and a real model-provider key, both blocked by the sandbox rather than by the design.
